A DEEP DIVE INTO SOFO

Second Order Forward-Mode Optimization

ZACH LIU

Work in progress, October 2026

CONTENTS

1 Recurrent Neural Networks 2
- 1.1 Backpropagation Through Time . . . 2
- 1.2 Why Curvature Matters for RNNs . . . 3

2 Automatic Differentiation for RNNs 3
- 2.1 Forward-Mode Tangent Propagation for RNNs . . . 3
- 2.2 Reverse-Mode Adjoint Propagation for RNNs (BPTT) . . . 4

3 Subspace Optimization Methods 5
- 3.1 Gaussian Sketches . . . 5
- 3.2 First-Order Subspace Descent . . . 5
- 3.3 Randomised Subspace Newton . . . 6
- 3.4 Connection to SOFO . . . 7

4 SOFO 7
- 4.1 Derivation . . . 7
- 4.2 Algorithm . . . 8
- 4.3 Complexity . . . 8
- 4.4 Application to RNNs . . . 9
- 4.5 What SOFO Guarantees . . . 9

5 SOFO-NPG: Forward-Mode Natural Policy Gradient 11
- 5.1 Setup . . . 11
- 5.2 The Surrogate's Own GGN Is Indefinite . . . 11
- 5.3 Fisher Equals GGN for Softmax Policies . . . 12
- 5.4 Sketched Fisher and the SOFO-NPG Update . . . 12
- 5.5 SOFO-TRPO . . . 13
- 5.6 Cross-Entropy Surrogate . . . 14
- 5.7 Relation to RTRL . . . 15
- 5.8 Limitations and Open Questions . . . 15

ABSTRACT

Notes on SOFO [1], a second-order optimiser that trains recurrent networks with batched forward-mode differentiation, and on SOFO-NPG, a proposed extension to policy optimisation. The Technical Background covers vanilla RNNs and why training them needs curvature, forward- and reverse-mode differentiation on the unrolled network, random subspace methods (SSD and RSN), and SOFO itself: a Gauss–Newton step in a random r-dimensional subspace whose curvature comes from r Jacobian-vector products, with memory independent of the horizon. Once r reaches the rank of the GGN, the sketched step is the exact Gauss–Newton step on its range, and the convergence rate depends on the sketch through $r/\mathrm{rank}(\mathbf{G})$ rather than r/P. The Research Direction proposes SOFO-NPG. For softmax policies the Fisher equals the GGN of the cross-entropy, so the same sketch gives a natural policy gradient, a closed-form trust-region step (SOFO-TRPO) and a cross-entropy surrogate on which SOFO runs unchanged, aimed at recurrent policies over long horizons. General optimisation theory is in the companion optimisation notes and RL background in the companion RL notes. The guarantees subsection and SOFO-NPG are unpublished; SOFO-NPG has not been tested.

DISCLOSURE

These notes were written collaboratively with Claude (Anthropic). I sketched the high-level structure, and Claude and I edited each paragraph and sentence iteratively. I take responsibility for any errors.

Technical Background

This part covers the recurrent networks SOFO trains (§1), automatic differentiation on them (§2), random subspace methods (§3) and SOFO itself (§4). Optimisation theory, the generalised Gauss–Newton matrix and the Fisher information live in the companion optimisation notes (Parts I and IV there, in particular §15 and §16), which this document cites rather than repeats.

1 RECURRENT NEURAL NETWORKS

SOFO was built for vanilla RNNs, whose loss surfaces are badly conditioned and whose gradients cost memory linear in the horizon; this section fixes the model and shows where both problems come from.

A vanilla RNN with hidden state $\mathbf{h}_t \in \mathbb{R}^S$, recurrent weights $\mathbf{W}_{hh} \in \mathbb{R}^{S\times S}$, input weights $\mathbf{W}_{hx}$, output weights $\mathbf{W}_{oh}$ and bias $\mathbf{b}_h$ evolves as

$$\begin{aligned} \mathbf{a}_t &= \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{hx}\,\mathbf{x}_t + \mathbf{b}_h \\ \mathbf{h}_t &= \phi(\mathbf{a}_t) \\ \mathbf{o}_t &= \mathbf{W}_{oh}\,\mathbf{h}_t \end{aligned} \tag{1}$$

where ϕ is a pointwise nonlinearity (typically ReLU) and $\mathbf{a}_t$ is the pre-activation. The parameter vector $\boldsymbol{\theta} \in \mathbb{R}^P$ collects all trainable weights. With output $\mathbf{o}_t \in \mathbb{R}^D$ and horizon T, one sequence produces $N = DT$ scalar outputs (Figure 1).

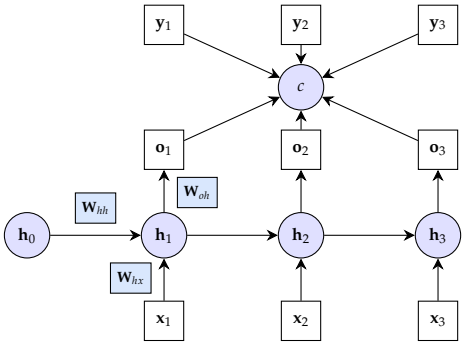


Figure 1: Unrolled computational graph of a vanilla RNN over $T = 3$ steps. The weights $\mathbf{W}_{hx}$, $\mathbf{W}_{hh}$ are shared across steps, $\mathbf{W}_{oh}$ reads out each hidden state, and the cost c sums the losses over all steps.

COST FUNCTION. For a minibatch of M sequences with outputs $\mathbf{o}_{mt}$ and targets $\mathbf{y}_{mt}$,

$$c(\boldsymbol{\theta}) \;=\; \frac{1}{M}\sum_{m=1}^{M}\sum_{t=1}^{T} \ell_{mt}\big(\mathbf{o}_{mt}(\boldsymbol{\theta}),\,\mathbf{y}_{mt}\big) \;=\; \frac{1}{M}\sum_{m=1}^{M}\sum_{i=1}^{N} \ell_{mi}(o_{mi}), \tag{2}$$

where each ℓ_{mi} is strongly convex in its first argument. This does not make c convex in $\boldsymbol{\theta}$. The cost c plays the role of the objective f of the optimisation notes.

1.1 Backpropagation Through Time

The standard way to compute $\nabla_{\boldsymbol{\theta}} c$ is backpropagation through time (BPTT) [2], which §2.2 derives from reverse-mode AD. Take one sequence ($M = 1$) with $c = \sum_t \ell_t(\mathbf{o}_t, \mathbf{y}_t)$. The output weights enter each output directly:

$$\frac{\partial c}{\partial \mathbf{W}_{oh}} = \sum_{t=1}^{T} \frac{\partial \ell_t}{\partial \mathbf{o}_t}\,\mathbf{h}_t^{\top}. \tag{3}$$

The recurrent weights are harder, because $\mathbf{h}_t$ depends on $\mathbf{W}_{hh}$ at step t and through every earlier hidden state. With the activation-derivative matrix

$$\mathbf{D}_t \;=\; \operatorname{diag}\big[\phi'([\mathbf{a}_t]_1),\,\ldots,\,\phi'([\mathbf{a}_t]_S)\big] \;\in\; \mathbb{R}^{S\times S} \tag{4}$$

(a binary mask for ReLU), the one-step Jacobian is $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = \mathbf{D}_t \mathbf{W}_{hh}$ and

$$\frac{\partial c}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \mathbf{D}_t\,\frac{\partial c}{\partial \mathbf{h}_t}\,\mathbf{h}_{t-1}^{\top}, \qquad \frac{\partial c}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^{T} \mathbf{D}_t\,\frac{\partial c}{\partial \mathbf{h}_t}\,\mathbf{x}_t^{\top}, \tag{5}$$

where $\partial c/\partial \mathbf{h}_t$ is propagated backward through $\mathbf{W}_{hh}^\top \mathbf{D}_{t+1}$ at each step. BPTT caches all T hidden states in the forward pass, $O(ST)$ memory, and then runs backward from $t = T$ to $t = 1$. The backward chain $\prod_j \mathbf{W}_{hh}^\top \mathbf{D}_j$ is the source of the pathologies listed in §2.2; SOFO avoids the backward pass by using forward-mode AD (§2).

1.2 Why Curvature Matters for RNNs

Vanishing and exploding gradients are the classical difficulty. Zucchet and Orvieto identify a second one that remains when the dynamics are stable: the longer a network remembers, the more sensitive its hidden state is to the recurrent parameters, so the loss sharpens. They call this the *curse of memory* [3].

Take the scalar linear RNN $h_{t+1} = \lambda h_t + x_{t+1}$ with $|\lambda| < 1$, started at $t = -\infty$, with i.i.d. unit-variance inputs. Then

$$
\begin{aligned}
h_t &= \sum_{k\geq 0} \lambda^k x_{t-k} && \text{(unroll the recursion)} \\
\frac{dh_t}{d\lambda} &= \sum_{k\geq 1} k\,\lambda^{k-1} x_{t-k} && \text{(differentiate term by term)} \\
\mathbb{E}[h_t^2] &= \sum_{k\geq 0} \lambda^{2k} = \frac{1}{1-\lambda^2} && (\mathbb{E}[x_i x_j] = \delta_{ij}) \\
\mathbb{E}\Big[\Big(\frac{dh_t}{d\lambda}\Big)^2\Big] &= \sum_{k\geq 1} k^2 \lambda^{2(k-1)} = \frac{1+\lambda^2}{(1-\lambda^2)^3} && (\textstyle\sum_{k\geq 1} k^2 q^{k-1} = \frac{1+q}{(1-q)^3} \text{ with } q = \lambda^2)
\end{aligned}
$$

As $|\lambda| \to 1$ the variance of the hidden state grows like $(1-\lambda^2)^{-1}$ but its sensitivity to λ like $(1-\lambda^2)^{-3}$: a small change in λ changes the whole trajectory, and the loss becomes sharp along λ although no gradient explodes. Correlated inputs make the growth faster [3, eq. 6]. Two consequences bear on the choice of optimiser [3, §2.3, §4.3].

- For a dense recurrent matrix $\mathbf{W}_{hh} = \mathbf{Q}\,\mathrm{diag}(\boldsymbol{\lambda})\,\mathbf{Q}^{-1}$, each entry moves every eigenvalue, so the sharp directions are spread over all entries of $\mathbf{W}_{hh}$. On a linear teacher–student task, dense and diagonal linear RNNs have similar Hessian spectra at the optimum, but only for the diagonal network are the top eigenvectors close to coordinate axes, which Adam's per-coordinate step sizes can follow; for the dense network Adam has to shrink all its step sizes.
- At a perfect fit the loss Hessian reduces to its Gauss–Newton term $\sum_t (d\mathbf{h}_t/d\boldsymbol{\theta})^\top \partial^2_{\mathbf{h}} \ell_t \, (d\mathbf{h}_t/d\boldsymbol{\theta})$, so the sharp directions are curvature that the generalised Gauss–Newton matrix (§15) represents and that SOFO sketches (§4).

Neuroscience models keep vanilla RNNs, without gates or diagonal reparameterisations, for biological plausibility [1]. Soo et al. find that such leaky vanilla RNNs, trained with BPTT on cognitive tasks with long-term dependencies, need many more iterations and fail two of the tasks within their training budget, while the same architecture learns them when training-time skip connections shorten the gradient path: the bottleneck is optimisation, not expressivity [4].

2 AUTOMATIC DIFFERENTIATION FOR RNNS

SOFO replaces backpropagation through time by forward-mode differentiation. This section specialises the two AD modes of the optimisation notes (§5) to the unrolled RNN, with the same notation: Jacobian $\mathbf{J}_f$, JVP $\mathbf{J}_f \mathbf{p}$, VJP $\mathbf{J}_f^\top \mathbf{s}$, tangents $\dot{\mathbf{x}}_k$ and adjoints $\bar{\mathbf{x}}_k$.

2.1 Forward-Mode Tangent Propagation for RNNs

Perturb the parameters along $\boldsymbol{\theta}(\epsilon) = \boldsymbol{\theta} + \epsilon\,\mathbf{p}$, where $\boldsymbol{\theta} = (\mathrm{vec}(\mathbf{W}_{hh}), \mathrm{vec}(\mathbf{W}_{hx}), \mathrm{vec}(\mathbf{W}_{oh}), \mathbf{b}_h)$ and $\mathbf{p} \in \mathbb{R}^P$. The weight tangents $\dot{\mathbf{W}}_{hh}, \dot{\mathbf{W}}_{hx}, \dot{\mathbf{W}}_{oh}, \dot{\mathbf{b}}_h$ are the constant blocks of $\mathbf{p}$ reshaped; only the state tangent $\dot{\mathbf{h}}_t := d\mathbf{h}_t/d\epsilon|_{\epsilon=0}$ evolves. Differentiating (1) with respect to ϵ:

$$
\dot{\mathbf{h}}_t \;=\; \mathbf{D}_t\big(\mathbf{W}_{hh}\,\dot{\mathbf{h}}_{t-1} + \dot{\mathbf{W}}_{hh}\,\mathbf{h}_{t-1} + \dot{\mathbf{W}}_{hx}\,\mathbf{x}_t + \dot{\mathbf{b}}_h\big), \qquad \dot{\mathbf{o}}_t = \mathbf{W}_{oh}\,\dot{\mathbf{h}}_t + \dot{\mathbf{W}}_{oh}\,\mathbf{h}_t. \tag{6}
$$

The first term carries the tangent through the recurrence; the others are the direct effect of the parameters used at step t. By the chain rule each scalar output tangent is $\dot{o}_i = \nabla_{\boldsymbol{\theta}}^\top o_i\,\mathbf{p}$, so stacking the output tangents over time gives the JVP

$$
\dot{\mathbf{o}} \;:=\; \begin{bmatrix} \dot{\mathbf{o}}_1 \\ \vdots \\ \dot{\mathbf{o}}_T \end{bmatrix} \;=\; \mathbf{J}\,\mathbf{p}, \qquad \mathbf{J} = \frac{\partial \mathbf{o}}{\partial \boldsymbol{\theta}^\top} \in \mathbb{R}^{N\times P},
$$

with $\mathbf{J}$ the Jacobian of Definition 18, computed without forming it (Figure 2).

MEMORY. The recursion (6) needs only $(\mathbf{h}_{t-1}, \dot{\mathbf{h}}_{t-1})$ and $\mathbf{x}_t$, so earlier pairs are discarded as soon as step t is done. One tangent costs $O(S)$ working memory, independent of T, whereas BPTT stores all T hidden states (§2.2).

BATCHED TANGENTS. Propagating r tangents at once replaces $\dot{\mathbf{h}}_t \in \mathbb{R}^S$ by a tangent matrix $\dot{\mathbf{H}}_t \in \mathbb{R}^{S\times r}$, each column following (6) with its own seed. Collecting the seeds as the columns of $\mathbf{\Theta} \in \mathbb{R}^{P\times r}$, the output tangents form

$$\mathbf{Y} \coloneqq \mathbf{J}\,\mathbf{\Theta} \in \mathbb{R}^{MN\times r} \tag{7}$$

over a minibatch of M sequences. SOFO draws $\mathbf{\Theta}$ with i.i.d. Gaussian entries and $r \ll P$ (between 1% and 5% of P in the SOFO experiments), so $\mathbf{Y}$ is the action of $\mathbf{J}$ on a random r-dimensional subspace; §4 extracts curvature from it. The r propagations are independent and run in parallel on a GPU [1].

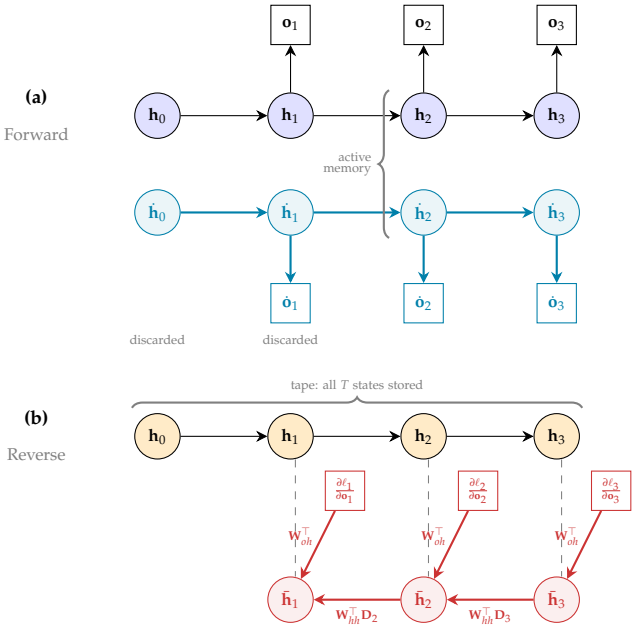


Figure 2: AD on the unrolled RNN of Figure 1. **(a)** Forward mode propagates tangents $\dot{\mathbf{h}}_t$ alongside hidden states by (6); only the current pair is in memory, $O(S)$. **(b)** Reverse mode propagates adjoints $\bar{\mathbf{h}}_t$ backward by (8). Each adjoint receives a direct signal from ℓ_t through $\mathbf{W}_{oh}^\top$ and a backpropagated one from $\bar{\mathbf{h}}_{t+1}$ through $\mathbf{W}_{hh}^\top \mathbf{D}_{t+1}$, and needs the stored primal $\mathbf{h}_t$ (dashed), so all T states are kept, $O(ST)$.

2.2 Reverse-Mode Adjoint Propagation for RNNs (BPTT)

Reverse mode on the same graph recovers BPTT. Define the adjoint $\bar{\mathbf{h}}_t \coloneqq \partial c/\partial \mathbf{h}_t \in \mathbb{R}^S$. The node $\mathbf{h}_t$ has two children, $\mathbf{o}_t$ and $\mathbf{h}_{t+1}$ (Figure 1), and the DAG rule (26) sums their contributions:

$$\begin{aligned}\bar{\mathbf{h}}_t &= \left(\frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t}\right)^\top \bar{\mathbf{o}}_t + \left(\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}\right)^\top \bar{\mathbf{h}}_{t+1} \qquad \text{(DAG rule)} \\ &= \mathbf{W}_{oh}^\top \frac{\partial \ell_t}{\partial \mathbf{o}_t} + \mathbf{W}_{hh}^\top \mathbf{D}_{t+1}\,\bar{\mathbf{h}}_{t+1} \qquad (\bar{\mathbf{o}}_t = \partial \ell_t/\partial \mathbf{o}_t,\ \partial \mathbf{h}_{t+1}/\partial \mathbf{h}_t = \mathbf{D}_{t+1}\mathbf{W}_{hh},\ \mathbf{D}_{t+1}^\top = \mathbf{D}_{t+1})\end{aligned} \tag{8}$$

The adjoint of the pre-activation is $\bar{\mathbf{a}}_t = \mathbf{D}_t \bar{\mathbf{h}}_t$, and each step contributes an outer product, $\partial c/\partial \mathbf{W}_{hh} = \sum_t \bar{\mathbf{a}}_t\, \mathbf{h}_{t-1}^\top$, which is (5). The recursion runs from $t = T$ down to $t = 1$ and needs every stored primal $\mathbf{h}_t$ (inside $\mathbf{D}_{t+1}$), hence the $O(ST)$ tape.

PATHOLOGIES OF BPTT. The backward chain $\prod_j \mathbf{W}_{hh}^\top \mathbf{D}_j$ causes three problems.

1. *Vanishing and exploding gradients.* Along an eigendirection of $\mathbf{W}_{hh}$ with eigenvalue λ_k, a contribution from $i - t$ steps back scales like $|\lambda_k|^{i-t}$, vanishing for $|\lambda_k| < 1$ and exploding for $|\lambda_k| > 1$. When the dynamics to be learned are chaotic, any RNN that reproduces them has a positive maximal Lyapunov exponent, and Mikhaeil et al. prove that the exact loss gradient then diverges exponentially in the horizon for vanilla RNNs, LSTMs, GRUs and PLRNNs; architectures with bounded gradients avoid this only by being unable to produce chaos [5]. The divergence is in the Jacobian products themselves, so forward-mode tangents

diverge in the same way and SOFO does not remove it. Their remedy is sparsely forced BPTT, which resets the hidden state to a value inferred from the data about once per predictability time $\ln 2/\lambda_{\max}$.
2. $O(ST)$ *memory.* Truncated BPTT cuts the backward pass at $\tau \ll T$ steps, reducing memory to $O(S\tau)$, but its gradients are biased and miss dependencies longer than τ.
3. *Curse of memory* (§1.2). Even with stable dynamics the loss sharpens along the recurrent weights, a curvature problem rather than a gradient-magnitude one.

3 SUBSPACE OPTIMIZATION METHODS

The damped GGN step (85) needs a $P \times P$ solve. Of the two ways around it (opener of Part IV of the optimisation notes), subspace methods keep the exact curvature and minimise the quadratic model only over a subspace. A Krylov subspace (CG, Hessian-free, TRPO) contains the gradient and adapts to the spectrum, but is built one curvature-vector product at a time, and for an RNN each product needs a reverse pass through time. A random subspace (SSD, RSN, SOFO) is drawn independently of the gradient and the curvature, and is built in parallel, with one forward-mode product per column for SOFO. This section develops the random family; SOFO is its second-order member.

3.1 Gaussian Sketches

Every method below draws a sketch $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ whose columns span the subspace. SOFO uses Gaussian sketches, and its analysis (§4.5) rests on two of their properties: rotational invariance and full rank almost surely.

Definition 1 (Gaussian sketch)**.** A *Gaussian sketch* is a random matrix $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ with entries $\Theta_{pi} \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$; equivalently, its columns $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_r$ are i.i.d. $\mathcal{N}(\mathbf{0}, \mathbf{I}_P)$. Since $\mathbb{E}[\boldsymbol{\theta}_i \boldsymbol{\theta}_i^\top] = \mathbf{I}_P$, it satisfies $\mathbb{E}[\boldsymbol{\Theta}\boldsymbol{\Theta}^\top] = \sum_i \mathbb{E}[\boldsymbol{\theta}_i \boldsymbol{\theta}_i^\top] = r\,\mathbf{I}_P$.

Lemma 1 (Rotational invariance)**.** *Let $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ be a Gaussian sketch and $\mathbf{V} \in \mathbb{R}^{P\times P}$ orthogonal. Then $\mathbf{V}\boldsymbol{\Theta} \overset{d}{=} \boldsymbol{\Theta}$.*

Proof. Each column $\mathbf{V}\boldsymbol{\theta}_j$ is a linear map of $\boldsymbol{\theta}_j \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_P)$, hence $\mathcal{N}(\mathbf{0}, \mathbf{V}\mathbf{V}^\top) = \mathcal{N}(\mathbf{0}, \mathbf{I}_P)$, and the columns stay independent. □

Corollary 2 (Projected Gaussian sketches are Gaussian)**.** *Let $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ be a Gaussian sketch and $\mathbf{U} \in \mathbb{R}^{P\times s}$ have orthonormal columns. Then $\mathbf{A} := \mathbf{U}^\top\boldsymbol{\Theta} \in \mathbb{R}^{s\times r}$ has i.i.d. $\mathcal{N}(0,1)$ entries, whatever subspace $\mathbf{U}$ spans.*

Proof. Each column $\mathbf{U}^\top\boldsymbol{\theta}_j$ is $\mathcal{N}(\mathbf{0}, \mathbf{U}^\top\mathbf{U}) = \mathcal{N}(\mathbf{0}, \mathbf{I}_s)$, and the columns stay independent. □

Lemma 3 (Full rank almost surely)**.** *A matrix $\boldsymbol{\Theta} \in \mathbb{R}^{n\times r}$ with i.i.d. $\mathcal{N}(0,1)$ entries has $\operatorname{rank}(\boldsymbol{\Theta}) = \min(n, r)$ with probability 1.*

Proof. Rank deficiency forces some $\min(n,r) \times \min(n,r)$ minor to vanish. Each minor is a nonzero polynomial in the entries, so its zero set has Lebesgue measure zero, and the Gaussian distribution is absolutely continuous. □

Corollary 4 (Sketched PSD matrices keep their rank)**.** *Let $\mathbf{B} \in \mathbb{R}^{P\times P}$ be symmetric PSD with rank s and $\boldsymbol{\Theta}$ a Gaussian sketch. Then $\operatorname{rank}(\boldsymbol{\Theta}^\top\mathbf{B}\boldsymbol{\Theta}) = \min(r, s)$ almost surely.*

Proof. With the thin eigendecomposition $\mathbf{B} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$, $\mathbf{U} \in \mathbb{R}^{P\times s}$, $\boldsymbol{\Lambda} \succ 0$, we have $\boldsymbol{\Theta}^\top\mathbf{B}\boldsymbol{\Theta} = (\mathbf{U}^\top\boldsymbol{\Theta})^\top\boldsymbol{\Lambda}(\mathbf{U}^\top\boldsymbol{\Theta})$, whose rank is that of $\mathbf{U}^\top\boldsymbol{\Theta}$. That matrix is Gaussian by Corollary 2, so its rank is $\min(s, r)$ a.s. by Lemma 3. □

So the $r \times r$ sketched curvature in RSN and SOFO is positive definite a.s. when $r \le s$, and singular when $r > s$, where the pseudoinverse of §2 replaces the inverse.

3.2 First-Order Subspace Descent

Stochastic subspace descent (SSD) [6] projects the gradient onto a random subspace at each step:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\,\boldsymbol{\Theta}_k\boldsymbol{\Theta}_k^\top\nabla f(\boldsymbol{\theta}_k) = \boldsymbol{\theta}_k - \alpha\sum_{i=1}^{r}\left([\boldsymbol{\Theta}_k]_i^\top\nabla f(\boldsymbol{\theta}_k)\right)[\boldsymbol{\Theta}_k]_i, \tag{9}$$

which needs only the r directional derivatives $[\boldsymbol{\Theta}_k]_i^\top\nabla f$, one forward-mode pass each, and never the full gradient. The analysis assumes the following.

Definition 2 (Sketch assumption (Ao) [6])**.** The sketches $\boldsymbol{\Theta}_k \in \mathbb{R}^{P\times r}$ are i.i.d. with

$$\mathbb{E}\left[\boldsymbol{\Theta}_k\boldsymbol{\Theta}_k^\top\right] = \mathbf{I}_P, \tag{Ao-1}$$

$$\boldsymbol{\Theta}_k^\top\boldsymbol{\Theta}_k = \tfrac{P}{r}\,\mathbf{I}_r \quad \text{for every realisation.} \tag{Ao-2}$$

(Ao-1) makes $\Theta_k\Theta_k^\top \nabla f$ an unbiased estimate of ∇f. The scaled Haar sketch $\sqrt{P/r}\,\mathbf{Q}\mathbf{I}_{P\times r}$, the first r columns of a uniformly random orthogonal matrix, satisfies both. A Gaussian sketch satisfies neither exactly: $\mathbb{E}[\Theta\Theta^\top] = r\mathbf{I}_P$, a factor SOFO absorbs into its step size, and $\Theta^\top\Theta$ is random.

Lemma 5 (SSD gradient-sketch MSE). *Under (Ao), with* $\mathbf{g} := \nabla f(\theta_k)$,

$$\mathbb{E}\left[\|\Theta_k\Theta_k^\top \mathbf{g} - \mathbf{g}\|^2\right] = \left(\frac{P}{r} - 1\right)\|\mathbf{g}\|^2. \tag{10}$$

Proof. The estimate is unbiased, so the MSE is $\mathbb{E}\|\Theta\Theta^\top\mathbf{g}\|^2 - \|\mathbf{g}\|^2$, and

$$\mathbb{E}\|\Theta\Theta^\top\mathbf{g}\|^2 = \mathbf{g}^\top\mathbb{E}\left[\Theta\,\Theta^\top\Theta\,\Theta^\top\right]\mathbf{g} = \frac{P}{r}\,\mathbf{g}^\top\mathbb{E}\left[\Theta\Theta^\top\right]\mathbf{g} = \frac{P}{r}\,\|\mathbf{g}\|^2,$$

using (Ao-2) and then (Ao-1). □

At $r/P = 1\%$ the factor $P/r - 1$ is 99: the estimate is very noisy but unbiased, and the expected descent it produces, $\mathbb{E}\,\mathbf{g}^\top\Theta\Theta^\top\mathbf{g} = \|\mathbf{g}\|^2$, is that of the full gradient. For L-smooth f satisfying PL with constant μ, step $\alpha = r/(PL)$ gives $\mathbb{E}f(\theta_k) - f^* \le (1 - r\mu/(PL))^k(f(\theta_0) - f^*)$ [6, Cor. 1]: the gradient-descent rate of Theorem 27 slowed by the factor r/P.

CONNECTION TO SOFO. The vector of directional derivatives $\mathbf{C} := \Theta^\top\nabla f \in \mathbb{R}^r$ is what SOFO's batched forward pass produces alongside $\mathbf{Y}$ (7), so SOFO computes SSD's gradient sketch and a curvature sketch in one sweep (§4).

3.3 Randomised Subspace Newton

Randomised subspace Newton (RSN) [7] replaces the projected gradient by the Newton step restricted to the subspace. Minimise the local model (27) with $\mathbf{B}_k = \mathbf{H}_k := \mathbf{H}_\theta(\theta_k)$ over steps $\mathbf{d} = \Theta_k\mathbf{h}$, $\mathbf{h} \in \mathbb{R}^r$, writing $\mathbf{g}_k := \nabla f(\theta_k)$:

$$M_k(\Theta_k\mathbf{h}) = f(\theta_k) + (\Theta_k^\top\mathbf{g}_k)^\top\mathbf{h} + \tfrac{1}{2}\,\mathbf{h}^\top(\Theta_k^\top\mathbf{H}_k\Theta_k)\mathbf{h} \quad \text{(substitute } \mathbf{d} = \Theta_k\mathbf{h}\text{)}$$

$$\mathbf{h}^* = -(\Theta_k^\top\mathbf{H}_k\Theta_k)^+\Theta_k^\top\mathbf{g}_k. \quad \text{(stationarity in } \mathbf{h}\text{)}$$

RSN takes this step scaled by $1/\hat{L}$, with $\hat{L}$ the relative smoothness constant below:

$$\theta_{k+1} \;=\; \theta_k \;-\; \frac{1}{\hat{L}}\,\Theta_k(\Theta_k^\top\mathbf{H}_k\Theta_k)^+\Theta_k^\top\mathbf{g}_k. \tag{11}$$

The $r \times r$ *sketched Hessian* $\Theta_k^\top\mathbf{H}_k\Theta_k$ costs r Hessian-vector products, each a nested forward-over-reverse pass (Algorithm 1).

Lemma 6 (RSN as a projected Newton step [7, Lemma 4]). *Suppose* $\mathbf{g}_k \in \mathrm{Range}(\mathbf{H}_k)$ *and let* $\mathbf{n}_k := -\mathbf{H}_k^+\mathbf{g}_k$ *be the Newton direction. Then*

$$\mathbf{P}_k \;:=\; \Theta_k(\Theta_k^\top\mathbf{H}_k\Theta_k)^+\Theta_k^\top\mathbf{H}_k \tag{12}$$

is the $\mathbf{H}_k$*-orthogonal projection onto* $\mathrm{Range}(\Theta_k)$ *(Lemma 11), and* $\theta_{k+1} = \theta_k + \frac{1}{\hat{L}}\,\mathbf{P}_k\mathbf{n}_k$*: the RSN step is the point of the sketch subspace closest to the full Newton step in the* $\mathbf{H}_k$*-norm (Figure 3).*

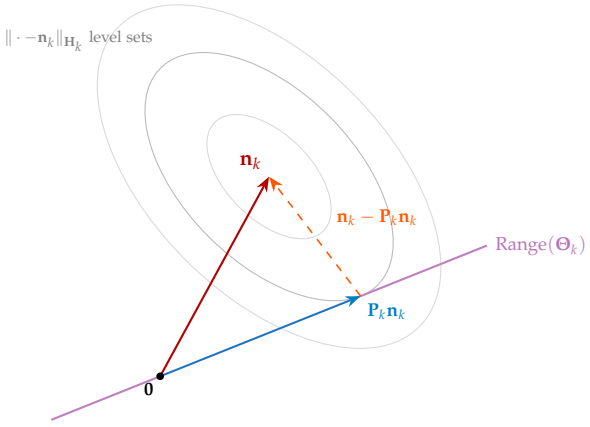


Figure 3: RSN projects the Newton direction $\mathbf{n}_k$ (red) onto the sketch subspace $\mathrm{Range}(\Theta_k)$. The projection $\mathbf{P}_k\mathbf{n}_k$ (blue) is the point of the subspace closest to $\mathbf{n}_k$ in the $\mathbf{H}_k$-norm, where a level set of $\|\cdot - \mathbf{n}_k\|_{\mathbf{H}_k}$ (grey ellipses) touches the subspace. The residual (orange) is $\mathbf{H}_k$-orthogonal to the subspace but not perpendicular to it in the Euclidean sense, so $\mathbf{P}_k\mathbf{n}_k$ is not the Euclidean projection (compare Figure 4 of the optimisation notes).

CONVERGENCE. RSN assumes f convex with $\mathbf{g}(\boldsymbol{\theta}) \in \mathrm{Range}(\mathbf{H}(\boldsymbol{\theta}))$, and *relatively smooth and convex*: there are $\hat{L} \geq \hat{\mu} > 0$ with

$$\frac{\hat{\mu}}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2_{\mathbf{H}(\boldsymbol{\theta})} \;\leq\; f(\boldsymbol{\theta}') - f(\boldsymbol{\theta}) - \mathbf{g}(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta}) \;\leq\; \frac{\hat{L}}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2_{\mathbf{H}(\boldsymbol{\theta})}$$

on the sublevel set $Q = \{f \leq f(\boldsymbol{\theta}_0)\}$; for a convex quadratic $\hat{L} = \hat{\mu} = 1$. With the *sketched condition number*

$$\rho \;\coloneqq\; \min_{\boldsymbol{\theta}\in Q} \lambda^+_{\min}\Big(\mathbf{H}^{1/2}\,\mathbb{E}\big[\boldsymbol{\Theta}(\boldsymbol{\Theta}^\top\mathbf{H}\boldsymbol{\Theta})^+\boldsymbol{\Theta}^\top\big]\,\mathbf{H}^{1/2}\Big) \;\in\; (0,1],$$

RSN satisfies $\mathbb{E}f(\boldsymbol{\theta}_k) - f^* \leq (1 - \rho\hat{\mu}/\hat{L})^k(f(\boldsymbol{\theta}_0) - f^*)$ [7, Thm. 2], and $\rho > 0$ whenever $\mathbb{E}[\boldsymbol{\Theta}\boldsymbol{\Theta}^\top]$ is invertible, as for Gaussian sketches [7, Lemma 7]. The ordinary condition number enters only through $\hat{L}/\hat{\mu}$; the sketch enters only through ρ, which §4.5 computes for SOFO.

3.4 Connection to SOFO

SOFO is RSN applied to a composite objective $f(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\phi}(\boldsymbol{\theta}))$ with the Hessian replaced by the GGN $\mathbf{G} = \mathbf{J}^\top\mathbf{H}_{\boldsymbol{\phi}}\mathbf{J}$. The factored form turns the sketched curvature into $\mathbf{Y}^\top\mathbf{H}_{\boldsymbol{\phi}}\mathbf{Y}$ with $\mathbf{Y} = \mathbf{J}\boldsymbol{\Theta}$, which needs r JVPs and no reverse pass, and the gradient sketch $\mathbf{C}$ of SSD comes out of the same sweep.

4 SOFO

Throughout, $\mathbf{H}_{\boldsymbol{\theta}}$ is the parameter-space Hessian of f, $\mathbf{H}_{\boldsymbol{\phi}}$ the output-space Hessian of the loss (Definition 20) and $\mathbf{G} = \mathbf{J}^\top\mathbf{H}_{\boldsymbol{\phi}}\mathbf{J}$ the GGN (§15).

4.1 Derivation

SOFO approximates the damped GGN step (85), which is infeasible at scale: $\mathbf{G}$ is $P \times P$. Four steps make it tractable.

RESTRICT TO A RANDOM SUBSPACE. As in RSN, write $\boldsymbol{\theta} = \boldsymbol{\theta}_k + \boldsymbol{\Theta}\mathbf{h}$ with $\mathbf{h} \in \mathbb{R}^r$ and expand f to second order, with $\mathbf{g}_k \coloneqq \nabla f(\boldsymbol{\theta}_k)$:

$$\begin{aligned} f(\boldsymbol{\theta}_k + \boldsymbol{\Theta}\mathbf{h}) &\approx f(\boldsymbol{\theta}_k) + \mathbf{g}_k^\top\boldsymbol{\Theta}\mathbf{h} + \tfrac{1}{2}(\boldsymbol{\Theta}\mathbf{h})^\top\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}_k)(\boldsymbol{\Theta}\mathbf{h}) && \text{(second-order Taylor)} \\ &= f(\boldsymbol{\theta}_k) + (\boldsymbol{\Theta}^\top\mathbf{g}_k)^\top\mathbf{h} + \tfrac{1}{2}\,\mathbf{h}^\top\big(\boldsymbol{\Theta}^\top\mathbf{H}_{\boldsymbol{\theta}}(\boldsymbol{\theta}_k)\boldsymbol{\Theta}\big)\mathbf{h}. && \text{(regroup)} \end{aligned} \tag{13}$$

The $P \times P$ solve becomes an $r \times r$ one, but the sketched Hessian still needs r Hessian-vector products.

REPLACE THE HESSIAN BY THE GGN. The Hessian splits as $\mathbf{H}_{\boldsymbol{\theta}} = \mathbf{G} + \mathbf{S}$ (81). Dropping $\mathbf{S}$ gives the PSD sketch $\boldsymbol{\Theta}^\top\mathbf{G}\boldsymbol{\Theta}$; this is exact when $\mathbf{S} = \mathbf{0}$, as for piecewise-linear activations, and otherwise the damping below keeps the step where the GGN model is accurate.

FACTOR THROUGH THE OUTPUTS. Substituting $\mathbf{G} = \mathbf{J}^\top\mathbf{H}_{\boldsymbol{\phi}}\mathbf{J}$:

$$\tilde{\mathbf{G}} \;\coloneqq\; \boldsymbol{\Theta}^\top\mathbf{J}^\top\mathbf{H}_{\boldsymbol{\phi}}\,\mathbf{J}\boldsymbol{\Theta} \;=\; (\mathbf{J}\boldsymbol{\Theta})^\top\mathbf{H}_{\boldsymbol{\phi}}\,(\mathbf{J}\boldsymbol{\Theta}) \;=\; \mathbf{Y}^\top\mathbf{H}_{\boldsymbol{\phi}}\,\mathbf{Y} \;\in\; \mathbb{R}^{r\times r}, \tag{14}$$

with $\mathbf{Y} = \mathbf{J}\boldsymbol{\Theta}$ from (7). Each column of $\mathbf{Y}$ is one JVP, and $\mathbf{H}_{\boldsymbol{\phi}}$ is applied analytically (§4.2): the r Hessian-vector products become r forward passes.

GRADIENT SKETCH FROM THE SAME SWEEP. The linear term of (13) is the gradient sketch, which by the chain rule is

$$\mathbf{C} \;\coloneqq\; \boldsymbol{\Theta}^\top\nabla f \;=\; \boldsymbol{\Theta}^\top\mathbf{J}^\top\nabla_{\boldsymbol{\phi}}\mathcal{L} \;=\; \mathbf{Y}^\top\nabla_{\boldsymbol{\phi}}\mathcal{L} \;\in\; \mathbb{R}^r, \tag{15}$$

the tangent batch of the loss itself, accumulated at no extra cost.

RELATIVE DAMPING. SOFO damps with $\gamma = \lambda\, s_{\max}(\tilde{\mathbf{G}})$, a dimensionless $\lambda > 0$ times the largest eigenvalue of the sketched GGN, which makes the step invariant to rescaling the loss (§14.5):

$$\gamma \;\coloneqq\; \lambda\, s_{\max}(\tilde{\mathbf{G}}). \tag{16}$$

Minimising the damped subspace model $q(\mathbf{h}) = \mathbf{C}^\top\mathbf{h} + \frac{1}{2}\mathbf{h}^\top(\tilde{\mathbf{G}} + \gamma\mathbf{I})\mathbf{h}$, which is strictly convex, gives $\mathbf{h}^* = -(\tilde{\mathbf{G}} + \gamma\mathbf{I})^{-1}\mathbf{C}$. With the eigendecomposition $\tilde{\mathbf{G}} = \mathbf{U}\mathbf{S}\mathbf{U}^\top$, the inverse is diagonal in the eigenbasis:

$$\begin{aligned} \big(\tilde{\mathbf{G}} + \gamma\,\mathbf{I}\big)^{-1} &= \big(\mathbf{U}\mathbf{S}\mathbf{U}^\top + \gamma\,\mathbf{U}\mathbf{U}^\top\big)^{-1} && (\mathbf{U}\mathbf{U}^\top = \mathbf{I}) \\ &= \mathbf{U}(\mathbf{S} + \gamma\,\mathbf{I})^{-1}\mathbf{U}^\top. && (\mathbf{S} + \gamma\mathbf{I} \text{ diagonal}) \end{aligned}$$

Lifting the step back to $\mathbb{R}^P$ with a learning rate η gives the SOFO update

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k \;-\; \eta\, \boldsymbol{\Theta}\, \mathbf{U}\big(\mathbf{S} + \lambda s_{\max}\, \mathbf{I}\big)^{-1} \mathbf{U}^\top \mathbf{C}. \tag{17}$$

With $\boldsymbol{\phi} = \boldsymbol{\theta}$ (so $\mathbf{J} = \mathbf{I}$ and $\mathbf{H}_{\boldsymbol{\phi}} = \mathbf{H}_{\boldsymbol{\theta}}$), $\gamma = 0$ and $\eta = 1/\hat{L}$, this is RSN (11). The composite structure changes the cost:

Quantity	RSN	SOFO
Curvature sketch	r HVPs, $r \times$ (fwd + rev)	r JVPs, $r\times$ fwd only
Gradient sketch	one reverse pass	same forward sweep
RNN memory (horizon T)	$O(ST)$ tape per HVP	independent of T

4.2 Algorithm

Algorithm 1 SOFO: Second-Order Forward-mode Optimisation [1]

1: **input:** initial parameters $\boldsymbol{\theta}_0 \in \mathbb{R}^P$
2: **hyperparameters:** sketch dimension r, learning rate $\eta > 0$, relative damping $\lambda > 0$
3: **for** $k = 0, 1, 2, \ldots$ **do**
4: sample data minibatch $\mathbf{x}$
5: sample $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ with $\Theta_{pi} \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$ ▷ *fresh subspace*
6: $\{\boldsymbol{\phi},\, \mathbf{Y}\} \leftarrow \texttt{model_fwd}\big(\{\boldsymbol{\theta},\, \boldsymbol{\Theta}\},\, \mathbf{x}\big)$ ▷ *batched forward mode,* $\mathbf{Y} = \mathbf{J}\boldsymbol{\Theta}$
7: $\{f,\, \mathbf{C}\} \leftarrow \texttt{loss_fwd}\big(\{\boldsymbol{\phi},\, \mathbf{Y}\}\big)$ ▷ $\mathbf{C} = \boldsymbol{\Theta}^\top \nabla f$
8: $\tilde{\mathbf{G}} \leftarrow \mathbf{Y}^\top \mathbf{H}_{\boldsymbol{\phi}}\, \mathbf{Y}$ ▷ *sketched GGN* (14)
9: $\mathbf{U},\, \mathbf{S} \leftarrow \texttt{eigh}(\tilde{\mathbf{G}})$; $\quad s_{\max} \leftarrow \max_i S_{ii}$ ▷ $r \times r$, *cost* $O(r^3)$
10: $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k \;-\; \eta\, \boldsymbol{\Theta}\, \mathbf{U}\big(\mathbf{S} + \lambda s_{\max}\, \mathbf{I}\big)^{-1} \mathbf{U}^\top \mathbf{C}$ ▷ *SOFO update* (17)
11: **end for**

A fresh Gaussian $\boldsymbol{\Theta}$ is drawn at every iteration; unlike a Haar sketch it needs no QR factorisation and costs $O(Pr)$ to sample. Line 6 propagates the r tangents through the network. Line 8 applies $\mathbf{H}_{\boldsymbol{\phi}}$ analytically; for losses that sum over outputs it is block diagonal with cheap blocks:

Loss	Block of $\mathbf{H}_{\boldsymbol{\phi}}$	Structure
Squared error	$\mathbf{I}$	identity
Binary cross-entropy	$\operatorname{diag}\big(\sigma(\boldsymbol{\phi}) \odot (1 - \sigma(\boldsymbol{\phi}))\big)$	diagonal
Categorical cross-entropy	$\operatorname{diag}(\mathbf{p}_t) - \mathbf{p}_t \mathbf{p}_t^\top$	diagonal plus rank one

When the loss is a sum over output blocks, $\tilde{\mathbf{G}}$ and $\mathbf{C}$ are sums too and can be accumulated block by block as the forward pass produces them, without storing $\mathbf{Y}$; §4.4 does this over time for RNNs.

4.3 Complexity

Per iteration, for a minibatch of M sequences of length T with D outputs per step:

Operation	Cost	Notes
Sample $\boldsymbol{\Theta}$	$O(Pr)$	memory and time
Batched forward pass ($\mathbf{Y}$, $\mathbf{C}$)	r JVPs	no reverse pass
Accumulate $\tilde{\mathbf{G}}$	$O(MT(D^2 r + Dr^2))$	one $D \times D$ block per output step
Eigendecomposition of $\tilde{\mathbf{G}}$	$O(r^3)$	small for r in the hundreds
Parameter update	$O(Pr)$	matrix-vector product

The r JVPs cost about r forward evaluations in total work, but they are independent and batched on the GPU, and SOFO's wall-clock time per iteration is a small constant (between 1.5 and 3) times Adam's [1, App. C].

4.4 Application to RNNs

For an RNN the tangent batch $\dot{\mathbf{H}}_t \in \mathbb{R}^{S\times r}$ follows (6) column by column, and the output tangents are $\dot{\mathbf{O}}_{mt} \in \mathbb{R}^{D\times r}$. The cost (2) is a sum over (m,t) and $\mathbf{H}_{\boldsymbol{\phi}}$ is block diagonal with $D\times D$ blocks $\mathbf{H}_{\boldsymbol{\phi},mt}$, so

$$\tilde{\mathbf{G}} \;=\; \frac{1}{M}\sum_m\sum_t \dot{\mathbf{O}}_{mt}^\top \mathbf{H}_{\boldsymbol{\phi},mt}\,\dot{\mathbf{O}}_{mt}, \qquad\qquad \mathbf{C} \;=\; \frac{1}{M}\sum_m\sum_t \dot{\mathbf{O}}_{mt}^\top \frac{\partial \ell_{mt}}{\partial \mathbf{o}_{mt}},$$

and at each step the $r\times r$ and r-dimensional updates are added and the state tangents of step $t-1$ discarded. The working memory is dominated by the parameter tangents of $\mathbf{W}_{hh}$, an $S\times S\times r$ array fixed for the sweep, and by the state tangents of the minibatch, $S\times M\times r$: $O(\max(S,M)\,S\,r)$, independent of T, against $O(S^2+SMT)$ for reverse mode. The time is $O(S^2MrT)$, r times that of reverse mode, recovered in wall-clock time by batching [1, App. C]. For a single sequence, with $P\approx S^2$:

Method	Working memory	Depends on T	Time per step	Time per iteration
BPTT	$O(S^2+ST)$	yes	$O(S^2)$	$O(S^2T)$
SOFO ($r\ll P$)	$O(S^2r)$	no	$O(S^2r)$	$O(S^2rT)$
RTRL ($r=P$)	$O(S^3)$	no	$O(S^4)$	$O(S^4T)$

Table 1: Memory and time per training iteration for a single-sequence vanilla RNN. S: state dimension; T: horizon; $P\approx S^2$; r: sketch dimension.

Real-time recurrent learning (RTRL) [8] is the limit of SOFO with the full basis $\boldsymbol{\Theta}=\mathbf{I}_P$ and no curvature: its sensitivity matrix $\partial\mathbf{h}_t/\partial\boldsymbol{\theta}^\top$ is the tangent batch for all P directions, and it needs P/r times SOFO's memory [1].

4.5 What SOFO Guarantees

The SOFO step is a random approximation to the GGN step. This subsection states what is known about it, for the undamped step $\gamma=0$ with $\eta=1/\hat{L}$; with damping the step is close to, but not equal to, the one analysed.

COLLAPSED FORM. Let $s:=\operatorname{rank}(\mathbf{G})\le\min(P,MN)$ (Lemma 51) and take the thin eigendecomposition $\mathbf{G}=\mathbf{U}\boldsymbol{\Sigma}\mathbf{U}^\top$ with $\mathbf{U}\in\mathbb{R}^{P\times s}$, $\boldsymbol{\Sigma}\succ 0$. The projected sketch $\mathbf{A}:=\mathbf{U}^\top\boldsymbol{\Theta}\in\mathbb{R}^{s\times r}$ is Gaussian (Corollary 2). If $\mathbf{H}_{\boldsymbol{\phi}}\succ 0$, the gradient lies in the range of $\mathbf{G}$:

$$\begin{aligned}\operatorname{Range}(\mathbf{G}) &= \operatorname{Range}\big((\mathbf{H}_{\boldsymbol{\phi}}^{1/2}\mathbf{J})^\top\mathbf{H}_{\boldsymbol{\phi}}^{1/2}\mathbf{J}\big) = \operatorname{Range}(\mathbf{J}^\top\mathbf{H}_{\boldsymbol{\phi}}^{1/2}) = \operatorname{Range}(\mathbf{J}^\top) && (\operatorname{Range}(\mathbf{B}^\top\mathbf{B})=\operatorname{Range}(\mathbf{B}^\top),\ \mathbf{H}_{\boldsymbol{\phi}}^{1/2}\text{ invertible})\\ \nabla f &= \mathbf{J}^\top\nabla_{\boldsymbol{\phi}}\mathcal{L}\in\operatorname{Range}(\mathbf{J}^\top) && (\text{chain rule})\end{aligned}$$

so $\nabla f=\mathbf{U}\mathbf{g}_\parallel$ with $\mathbf{g}_\parallel:=\mathbf{U}^\top\nabla f\in\mathbb{R}^s$, and both sketched quantities depend on $\boldsymbol{\Theta}$ only through $\mathbf{A}$:

$$\tilde{\mathbf{G}}=\boldsymbol{\Theta}^\top\mathbf{U}\boldsymbol{\Sigma}\mathbf{U}^\top\boldsymbol{\Theta}=\mathbf{A}^\top\boldsymbol{\Sigma}\mathbf{A}, \qquad \mathbf{C}=\boldsymbol{\Theta}^\top\mathbf{U}\mathbf{g}_\parallel=\mathbf{A}^\top\mathbf{g}_\parallel. \tag{18}$$

Theorem 7 (Exact Newton step for $r\ge\operatorname{rank}\mathbf{G}$). *Suppose $\mathbf{H}_{\boldsymbol{\phi}}\succ 0$, $r\ge s$ and $\gamma=0$. Then almost surely the sketched Newton decrement equals the full one,*

$$\mathbf{C}^\top\tilde{\mathbf{G}}^+\mathbf{C} \;=\; \nabla f^\top\mathbf{G}^+\nabla f, \tag{19}$$

and the component of the SOFO direction $\boldsymbol{\Theta}\tilde{\mathbf{G}}^+\mathbf{C}$ in $\operatorname{Range}(\mathbf{G})$ is the GGN Newton direction $\mathbf{G}^+\nabla f$.

Proof. For $r\ge s$, $\mathbf{A}$ has full row rank a.s. (Lemma 3), so $\mathbf{B}:=\boldsymbol{\Sigma}^{1/2}\mathbf{A}$ has column space $\mathbb{R}^s$ and its orthogonal projector $\mathbf{B}(\mathbf{B}^\top\mathbf{B})^+\mathbf{B}^\top$ (Lemma 10) is $\mathbf{I}_s$. Hence

$$\begin{aligned}\mathbf{A}(\mathbf{A}^\top\boldsymbol{\Sigma}\mathbf{A})^+\mathbf{A}^\top &= \boldsymbol{\Sigma}^{-1/2}\,\mathbf{B}(\mathbf{B}^\top\mathbf{B})^+\mathbf{B}^\top\,\boldsymbol{\Sigma}^{-1/2} && (\mathbf{A}=\boldsymbol{\Sigma}^{-1/2}\mathbf{B})\\ &= \boldsymbol{\Sigma}^{-1}. && (\text{projector is }\mathbf{I}_s)\end{aligned} \tag{20}$$

By (18), $\mathbf{C}^\top\tilde{\mathbf{G}}^+\mathbf{C}=\mathbf{g}_\parallel^\top\mathbf{A}(\mathbf{A}^\top\boldsymbol{\Sigma}\mathbf{A})^+\mathbf{A}^\top\mathbf{g}_\parallel=\mathbf{g}_\parallel^\top\boldsymbol{\Sigma}^{-1}\mathbf{g}_\parallel=\nabla f^\top\mathbf{G}^+\nabla f$, and the range component is $\mathbf{U}\mathbf{U}^\top\boldsymbol{\Theta}\tilde{\mathbf{G}}^+\mathbf{C}=\mathbf{U}\mathbf{A}(\mathbf{A}^\top\boldsymbol{\Sigma}\mathbf{A})^+\mathbf{A}^\top\mathbf{g}_\parallel=\mathbf{U}\boldsymbol{\Sigma}^{-1}\mathbf{g}_\parallel=\mathbf{G}^+\nabla f$. □

The rest of the step lies in $\operatorname{Null}(\mathbf{G})=\operatorname{Null}(\mathbf{J})$, so it is random but leaves the linearised outputs unchanged: on the GGN model, SOFO with $r\ge s$ takes the full Gauss–Newton step on every draw. What matters is r against the rank of $\mathbf{G}$, not against P.

Definition 3 (GGN-relative smoothness and convexity). f is $\hat{L}$-*GGN-relatively smooth* and $\hat{\mu}$-*GGN-relatively convex* on $Q = \{f \le f(\boldsymbol{\theta}_0)\}$ if for all $\boldsymbol{\theta}, \boldsymbol{\theta}' \in Q$

$$\frac{\hat{\mu}}{2}\,\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2_{\mathbf{G}(\boldsymbol{\theta})} \;\le\; f(\boldsymbol{\theta}') - f(\boldsymbol{\theta}) - \nabla f(\boldsymbol{\theta})^\top(\boldsymbol{\theta}' - \boldsymbol{\theta}) \;\le\; \frac{\hat{L}}{2}\,\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2_{\mathbf{G}(\boldsymbol{\theta})}.$$

This is RSN's assumption (§3.3) with $\mathbf{G}$ as the reference metric in place of $\mathbf{H}_{\boldsymbol{\theta}}$. For ReLU networks $\mathbf{S} = \mathbf{0}$ and $\mathbf{G} = \mathbf{H}_{\boldsymbol{\theta}}$ at every point; $\hat{L}$ and $\hat{\mu}$ then measure how much the curvature changes along a step.

SKETCHED CONDITION NUMBER. RSN's ρ with $\mathbf{G}$ in place of $\mathbf{H}$ is

$$\rho \;:=\; \min_{\boldsymbol{\theta}\in Q} \lambda^+_{\min}\Big(\mathbf{G}^{1/2}\,\mathbb{E}\big[\boldsymbol{\Theta}(\boldsymbol{\Theta}^\top\mathbf{G}\boldsymbol{\Theta})^+\boldsymbol{\Theta}^\top\big]\,\mathbf{G}^{1/2}\Big).$$

The substitution $\mathbf{B} = \boldsymbol{\Sigma}^{1/2}\mathbf{A}$ of the proof above reduces the sandwiched matrix to $\mathbf{U}\,\mathbb{E}[\boldsymbol{\Pi}_r]\,\mathbf{U}^\top$, where $\boldsymbol{\Pi}_r \in \mathbb{R}^{s\times s}$ is the orthogonal projector onto the column space of $\boldsymbol{\Sigma}^{1/2}\mathbf{A}$, so ρ is the smallest eigenvalue of $\mathbb{E}[\boldsymbol{\Pi}_r]$, a quantity in the s-dimensional curvature space.

Proposition 8 (Values of ρ). (i) *If $r \ge s$, then $\rho = 1$.* (ii) *If $r < s$ and $\boldsymbol{\Sigma} = \sigma\mathbf{I}_s$, then $\rho = r/s$.*

In (i), $\boldsymbol{\Pi}_r = \mathbf{I}_s$ a.s. by the proof of Theorem 7. In (ii), rotational invariance makes $\mathbb{E}[\boldsymbol{\Pi}_r]$ commute with every orthogonal matrix, so it is a multiple of $\mathbf{I}_s$, and its trace is $\operatorname{tr}\boldsymbol{\Pi}_r = r$ (Lemma 12).

Theorem 9 (SOFO linear convergence). *Let f be $\hat{L}$-GGN-relatively smooth and $\hat{\mu}$-GGN-relatively convex with $\rho > 0$, and run SOFO with $\gamma = 0$ and $\eta = 1/\hat{L}$. Then*

$$\mathbb{E}[f(\boldsymbol{\theta}_k)] - f^* \;\le\; \Big(1 - \frac{\hat{\mu}\rho}{\hat{L}}\Big)^k (f(\boldsymbol{\theta}_0) - f^*). \tag{21}$$

This is the linear-convergence template of the optimisation notes (Lemma 20) in the GGN norm. Relative smoothness evaluated at the step gives the expected decrease $\mathbb{E}f(\boldsymbol{\theta}_{k+1}) \le f(\boldsymbol{\theta}_k) - \frac{\rho}{2\hat{L}}\|\nabla f\|^2_{\mathbf{G}^+}$, and relative convexity gives the PL-type bound $\|\nabla f\|^2_{\mathbf{G}^+} \ge 2\hat{\mu}(f - f^*)$. Without relative convexity, the non-convex template (Lemma 22) gives $\min_{i<k}\mathbb{E}\|\nabla f(\boldsymbol{\theta}_i)\|^2_{\mathbf{G}^+} \le 2\hat{L}(f(\boldsymbol{\theta}_0) - f^*)/(\rho k)$. Both rates depend on the sketch through ρ, which compares r with s, not with P.

WHAT ρ DOES AND DOES NOT EXPLAIN. For $r < s$, $\boldsymbol{\Pi}_r$ is an orthogonal projector of rank r on $\mathbb{R}^s$, so

$$\begin{aligned} \operatorname{tr}\mathbb{E}[\boldsymbol{\Pi}_r] &= \mathbb{E}[\operatorname{tr}\boldsymbol{\Pi}_r] = r && \text{(Lemma 12)} \\ \rho = \lambda_{\min}\big(\mathbb{E}[\boldsymbol{\Pi}_r]\big) &\le \tfrac{1}{s}\operatorname{tr}\mathbb{E}[\boldsymbol{\Pi}_r] = \frac{r}{s}, && \text{(smallest eigenvalue at most the average)} \end{aligned}$$

with equality for isotropic curvature (Proposition 8). Whatever the spectrum, the rate of Theorem 9 degrades at least like r/s, and for a recurrent network s can grow with the horizon (§17). So the theorem cannot by itself explain why $r/P \approx 1\%$ works in practice. It bounds progress along the least favoured direction of curvature space. With damping, the full damped step uses curvature only along the s_γ eigendirections above γ and is an isotropic gradient step along the rest (§17), and measured spectra have few such directions (§18.1). The collapsed form suggests that a sketch with r somewhat above s_γ resolves them: with $\mathbf{a}_i^\top$ the rows of $\mathbf{A}$,

$$\tilde{\mathbf{G}} \;=\; \mathbf{A}^\top\boldsymbol{\Sigma}\mathbf{A} \;=\; \sum_{i=1}^{s}\sigma_i\,\mathbf{a}_i\mathbf{a}_i^\top,$$

which is dominated by the s_γ terms with large σ_i. Along the remaining directions SOFO's step is zero outside $\operatorname{Range}(\boldsymbol{\Theta})$, so it keeps only a random r-dimensional part of the gradient step there, as SSD does (§3.2). Whether s_γ rather than s governs SOFO's progress is not proved here; it is part of the open convergence question.

SKETCH NOISE AS STOCHASTIC-GRADIENT NOISE. The first-order part of SOFO is a stochastic gradient, and can be checked against the moment assumptions of stochastic-gradient theory (Assumption 2). Take the rescaled gradient sketch $\tilde{\mathbf{g}} := \frac{1}{r}\boldsymbol{\Theta}\boldsymbol{\Theta}^\top\mathbf{g} = \frac{1}{r}\boldsymbol{\Theta}\mathbf{C}$, which is unbiased since $\mathbb{E}[\boldsymbol{\Theta}\boldsymbol{\Theta}^\top] = r\mathbf{I}_P$. Its second moment needs $\mathbb{E}[(\boldsymbol{\Theta}\boldsymbol{\Theta}^\top)^2]$:

$$\begin{aligned} \mathbb{E}\big[(\boldsymbol{\Theta}\boldsymbol{\Theta}^\top)^2\big] &= \sum_i \mathbb{E}\big[\boldsymbol{\theta}_i\boldsymbol{\theta}_i^\top\boldsymbol{\theta}_i\boldsymbol{\theta}_i^\top\big] + \sum_{i\neq j}\mathbb{E}\big[\boldsymbol{\theta}_i\boldsymbol{\theta}_i^\top\big]\,\mathbb{E}\big[\boldsymbol{\theta}_j\boldsymbol{\theta}_j^\top\big] && \text{(expand } \boldsymbol{\Theta}\boldsymbol{\Theta}^\top = \textstyle\sum_i \boldsymbol{\theta}_i\boldsymbol{\theta}_i^\top\text{; independent columns)} \\ &= r(P+2)\,\mathbf{I}_P + r(r-1)\,\mathbf{I}_P && (\mathbb{E}[\|\boldsymbol{\theta}\|^2\theta_a\theta_b] = \delta_{ab}(\mathbb{E}\theta_a^4 + P - 1) = (P+2)\delta_{ab}) \\ &= r(r+P+1)\,\mathbf{I}_P, \end{aligned}$$

so that

$$\mathbb{E}\|\tilde{\mathbf{g}}\|^2 = \frac{1}{r^2}\,\mathbf{g}^\top\mathbb{E}\big[(\boldsymbol{\Theta}\boldsymbol{\Theta}^\top)^2\big]\mathbf{g} = \frac{r+P+1}{r}\,\|\mathbf{g}\|^2, \qquad \mathbb{V}[\tilde{\mathbf{g}}] = \frac{P+1}{r}\,\|\mathbf{g}\|^2.$$

With the exact gradient, Assumption 2 therefore holds with $\nu = \nu_G = 1$, $M = 0$ and $M_V = (P+1)/r$; a Haar sketch satisfying (Ao) gives $M_V = P/r - 1$ (Lemma 5). The noise vanishes at a stationary point, so a fixed step converges without a noise floor (Theorem 34 with $M = 0$). With a minibatch gradient $\mathbf{g}_B$ satisfying $\mathbb{E}\|\mathbf{g}_B\|^2 \le M + M_G\|\nabla f\|^2$, the same computation gives $\mathbb{E}\|\tilde{\mathbf{g}}\|^2 = \frac{r+P+1}{r}\,\mathbb{E}\|\mathbf{g}_B\|^2$, so the noise floor constant becomes $\frac{P+r+1}{r}\,M$. SOFO's actual direction $\mathbf{\Theta}(\tilde{\mathbf{G}} + \gamma\mathbf{I})^{-1}\mathbf{\Theta}^\top\mathbf{g}$ is a preconditioned sketch whose preconditioner, and damping $\gamma = \lambda s_{\max}(\tilde{\mathbf{G}})$, are built from the same $\mathbf{\Theta}$. Assumption 2(b) is verified in the notes for a preconditioner independent of the noise, so for SOFO it has to be checked directly; this is open.

Research Direction: SOFO-NPG

5 SOFO-NPG: FORWARD-MODE NATURAL POLICY GRADIENT

SOFO-NPG applies SOFO's forward-mode curvature sketch to the natural policy gradient. The target is recurrent policies over long horizons, where TRPO's conjugate-gradient solve needs a backward pass through the whole trajectory for every Fisher-vector product. Its possible advantages over TRPO are memory independent of the horizon, curvature products computed in parallel rather than in sequence, an exact solve within the subspace, and forward-in-time learning in the style of RTRL (§5.7). Each comes with a cost (§5.4), and all four are hypotheses to be tested.

5.1 Setup

A policy $\pi_{\boldsymbol{\theta}}(a \mid s)$ over a finite action set is a softmax of the network's logits $\boldsymbol{\phi}(s;\boldsymbol{\theta}) \in \mathbb{R}^{|A|}$. For a recurrent policy, s_t stands for the observation history summarised by the hidden state $\mathbf{h}_t$ of (1), and the logits are the network output at step t. Write $\boldsymbol{\phi}_t := \boldsymbol{\phi}(s_t;\boldsymbol{\theta})$, $\boldsymbol{\pi}_t := \mathrm{softmax}(\boldsymbol{\phi}_t)$ and $\mathbf{J}_t := \partial\boldsymbol{\phi}_t/\partial\boldsymbol{\theta}^\top \in \mathbb{R}^{|A|\times P}$.

The objective $J(\boldsymbol{\theta})$ is the expected return. By the policy gradient theorem with an advantage baseline (§4.2, §4.4 of the RL notes), its gradient is estimated from a rollout $(s_t, a_t, \hat{A}_t)_{t=1}^T$ by

$$\hat{\mathbf{g}} \;:=\; \frac{1}{T}\sum_{t=1}^{T} \hat{A}_t\,\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t). \tag{22}$$

The natural policy gradient [9] (§8 of the RL notes) preconditions this gradient by the policy Fisher

$$\mathbf{F}(\boldsymbol{\theta}) \;=\; \mathbb{E}_{s\sim d^\pi,\, a\sim\pi_{\boldsymbol{\theta}}(\cdot|s)}\big[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)\,\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a \mid s)^\top\big], \tag{23}$$

the local Hessian of the average KL divergence between successive policies (Proposition 48). TRPO [10] (§9 of the RL notes) maximises the linearised surrogate subject to $\frac{1}{2}\mathbf{d}^\top\mathbf{F}\mathbf{d} \le \delta$, whose solution is $\mathbf{d}^* = \sqrt{2\delta/(\mathbf{g}^\top\mathbf{F}^{-1}\mathbf{g})}\,\mathbf{F}^{-1}\mathbf{g}$ by the KL-ball derivation of §16.5 with the sign reversed for ascent. TRPO solves $\mathbf{F}\mathbf{x} = \mathbf{g}$ by conjugate gradient (Algorithm 5) and then backtracks until the true average KL is below δ.

RECURRENT POLICIES. Each CG iteration needs one Fisher-vector product. TRPO computes it as $\mathbf{J}^\top\mathbf{M}\mathbf{J}\mathbf{v}$, with $\mathbf{M}$ the Fisher of the action distribution in output space [10, App. C.1]: a GGN-vector product (Algorithm 2), one JVP and one VJP. For a recurrent policy the VJP backpropagates through all T steps of the rollout, so every product needs the $O(ST)$ tape (§2.2), and the CG iterations run one after another. SOFO-NPG replaces them with r forward-mode tangents propagated in one sweep, with memory independent of T.

5.2 The Surrogate's Own GGN Is Indefinite

SOFO cannot be run on the TRPO surrogate itself. At a single state, the surrogate's expectation over $a \sim \pi_{\text{old}}$ of $\frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\text{old}}(a|s)}A(s,a)$ is $L_s(\boldsymbol{\phi}) = \mathbf{A}^\top\boldsymbol{\pi}(\boldsymbol{\phi})$, with $\mathbf{A} \in \mathbb{R}^{|A|}$ the vector of advantages at s. It is linear in $\boldsymbol{\pi}$ but not in $\boldsymbol{\phi}$. With $\mathbf{u} := \boldsymbol{\pi}\odot\mathbf{A}$ and the softmax Jacobian $\partial\boldsymbol{\pi}/\partial\boldsymbol{\phi}^\top = \mathrm{diag}(\boldsymbol{\pi}) - \boldsymbol{\pi}\boldsymbol{\pi}^\top =: \mathbf{H}_s$:

$$\begin{aligned}
\nabla_{\boldsymbol{\phi}} L_s &= \mathbf{H}_s\mathbf{A} = \mathbf{u} - (\boldsymbol{\pi}^\top\mathbf{A})\,\boldsymbol{\pi} && \text{(chain rule)}\\
\nabla^2_{\boldsymbol{\phi}} L_s &= \mathrm{diag}(\mathbf{A})\,\mathbf{H}_s - \boldsymbol{\pi}\,(\mathbf{H}_s\mathbf{A})^\top - (\boldsymbol{\pi}^\top\mathbf{A})\,\mathbf{H}_s && \text{(product rule)}\\
&= \mathrm{diag}(\mathbf{u}) - \mathbf{u}\boldsymbol{\pi}^\top - \boldsymbol{\pi}\mathbf{u}^\top && \text{(at } \boldsymbol{\theta}_{\text{old}}\text{: } \boldsymbol{\pi}^\top\mathbf{A} = \mathbb{E}_{a\sim\pi_{\text{old}}}A(s,a) = 0)
\end{aligned}$$

Its diagonal entries are $\pi_a A(s,a)(1-2\pi_a)$, which for actions with $\pi_a < \frac{1}{2}$ have the sign of $A(s,a)$. Advantages average to zero, so they take both signs, and the output Hessian is indefinite in general. The sampled form $-\hat{A}_t \log \pi(a_t \mid s_t)$ has output Hessian $\hat{A}_t\mathbf{H}_{s_t}$, positive or negative semidefinite with the sign of $\hat{A}_t$, so its sum over

a rollout is indefinite as well. Either way $\mathbf{J}^\top(\nabla^2_{\boldsymbol{\phi}} L)\mathbf{J}$ is not a usable curvature matrix. In TRPO the quadratic term comes instead from the KL constraint, whose Hessian is the Fisher (23), and SOFO-NPG takes its curvature from there.

5.3 Fisher Equals GGN for Softmax Policies

For one state, $\log \pi(a \mid s) = \phi_a - \log \sum_{a'} e^{\phi_{a'}}$. Differentiating:

$$\nabla_{\boldsymbol{\phi}} \log \pi(a \mid s) = \mathbf{e}_a - \boldsymbol{\pi}_s \qquad (\partial_{\phi_{a'}} \log \textstyle\sum e^{\phi} = \pi_{a'})$$

$$\nabla_{\boldsymbol{\theta}} \log \pi(a \mid s) = \mathbf{J}_s^\top (\mathbf{e}_a - \boldsymbol{\pi}_s) \qquad \text{(chain rule)} \tag{24}$$

$$\mathbb{E}_{a\sim\boldsymbol{\pi}_s}\left[(\mathbf{e}_a - \boldsymbol{\pi}_s)(\mathbf{e}_a - \boldsymbol{\pi}_s)^\top\right] = \operatorname{diag}(\boldsymbol{\pi}_s) - \boldsymbol{\pi}_s\boldsymbol{\pi}_s^\top = \mathbf{H}_s \qquad (\mathbb{E}[\mathbf{e}_a\mathbf{e}_a^\top] = \operatorname{diag}(\boldsymbol{\pi}_s),\ \mathbb{E}[\mathbf{e}_a] = \boldsymbol{\pi}_s) \tag{25}$$

So the output-space Fisher is $\mathbf{H}_s$, the softmax Jacobian of §5.2, which is also the Hessian of the cross-entropy $-\log \pi(a \mid s)$ in $\boldsymbol{\phi}$ and does not depend on a: the categorical distribution is an exponential family with natural parameter $\boldsymbol{\phi}$ (§16). By the decomposition (89) and Theorem 49,

$$\mathbf{F}(\boldsymbol{\theta}) \;=\; \mathbb{E}_{s\sim d^\pi}\left[\mathbf{J}_s^\top \mathbf{H}_s \mathbf{J}_s\right], \tag{26}$$

the GGN of the cross-entropy loss with actions sampled from the policy. $\mathbf{H}_s$ has rank $|A| - 1$: $\mathbf{H}_s\mathbf{1} = \boldsymbol{\pi}_s - \boldsymbol{\pi}_s(\boldsymbol{\pi}_s^\top \mathbf{1}) = \mathbf{0}$, since the softmax is unchanged by $\boldsymbol{\phi} \mapsto \boldsymbol{\phi} + c\mathbf{1}$, and $\operatorname{Range}(\mathbf{H}_s) = \mathbf{1}^\perp$.

5.4 Sketched Fisher and the SOFO-NPG Update

Given a rollout of T steps under $\pi_{\boldsymbol{\theta}_k}$ and a Gaussian sketch $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$, batched forward mode gives the output tangents

$$\mathbf{Y}_t \;:=\; \mathbf{J}_t\boldsymbol{\Theta} \;\in\; \mathbb{R}^{|A|\times r} \tag{27}$$

for every step in one sweep (§4.4). From them,

$$\mathbf{y}_t := \mathbf{Y}_t^\top(\mathbf{e}_{a_t} - \boldsymbol{\pi}_t) = \boldsymbol{\Theta}^\top \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t) \qquad \text{(sketched score, (24))} \tag{28}$$

$$\tilde{\mathbf{F}} := \frac{1}{T}\sum_{t=1}^{T} \mathbf{Y}_t^\top \mathbf{H}_{s_t} \mathbf{Y}_t = \boldsymbol{\Theta}^\top \hat{\mathbf{F}} \boldsymbol{\Theta}, \qquad \hat{\mathbf{F}} := \frac{1}{T}\sum_{t=1}^{T} \mathbf{J}_t^\top \mathbf{H}_{s_t} \mathbf{J}_t \qquad \text{(sketched Fisher)} \tag{29}$$

$$\mathbf{C} := \frac{1}{T}\sum_{t=1}^{T} \hat{A}_t\, \mathbf{y}_t = \boldsymbol{\Theta}^\top \hat{\mathbf{g}} \qquad \text{(gradient sketch, (22))} \tag{30}$$

$\hat{\mathbf{F}}$ is the Fisher (26) with the state expectation replaced by an average over the rollout. The expectation over actions is done analytically through $\mathbf{H}_{s_t}$, the same choice as TRPO's analytic Fisher [10, §6]. The sampled outer product $\mathbf{y}_t\mathbf{y}_t^\top$ is an unbiased estimate of $\mathbf{Y}_t^\top \mathbf{H}_{s_t} \mathbf{Y}_t$, because on-policy actions are drawn from the model (§16.6), but it has higher variance. The step is SOFO's with the sign reversed, since J is maximised:

$$\boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k + \eta\, \boldsymbol{\Theta}\, \mathbf{U}(\mathbf{S} + \lambda s_{\max}\mathbf{I})^{-1}\mathbf{U}^\top \mathbf{C}, \qquad \tilde{\mathbf{F}} = \mathbf{U}\mathbf{S}\mathbf{U}^\top. \tag{31}$$

Algorithm 2 SOFO-NPG

hyperparameters: sketch dimension r, learning rate $\eta > 0$, relative damping $\lambda > 0$
for $k = 0, 1, 2, \ldots$ **do**
  roll out $\pi_{\boldsymbol{\theta}_k}$ for T steps; compute advantages $\hat{A}_t$
  sample $\boldsymbol{\Theta} \in \mathbb{R}^{P\times r}$ with $\Theta_{pi} \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$; $\tilde{\mathbf{F}} \leftarrow \mathbf{0}$, $\mathbf{C} \leftarrow \mathbf{0}$
  for $t = 1, \ldots, T$ **do**
    $\{\boldsymbol{\pi}_t,\ \mathbf{Y}_t\} \leftarrow \texttt{fwd}(\{\boldsymbol{\theta}_k,\ \boldsymbol{\Theta}\},\ s_t)$ ▷ *batched forward mode*, $\mathbf{Y}_t = \mathbf{J}_t\boldsymbol{\Theta}$
    $\tilde{\mathbf{F}} \mathrel{+}= \mathbf{Y}_t^\top(\operatorname{diag}(\boldsymbol{\pi}_t) - \boldsymbol{\pi}_t\boldsymbol{\pi}_t^\top)\mathbf{Y}_t / T$ ▷ (29)
    $\mathbf{C} \mathrel{+}= \hat{A}_t\, \mathbf{Y}_t^\top(\mathbf{e}_{a_t} - \boldsymbol{\pi}_t)/T$ ▷ (30)
  end for
  $\mathbf{U}, \mathbf{S} \leftarrow \texttt{eigh}(\tilde{\mathbf{F}})$; $s_{\max} \leftarrow \max_i S_{ii}$
  $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k + \eta\, \boldsymbol{\Theta}\, \mathbf{U}(\mathbf{S} + \lambda s_{\max}\mathbf{I})^{-1}\mathbf{U}^\top \mathbf{C}$ ▷ *ascent step* (31)
end for

WHEN THE SKETCH IS EXACT. Theorem 7 carries over with $\mathbf{G}$ replaced by $\hat{\mathbf{F}}$. Its proof needs the gradient in $\mathrm{Range}(\hat{\mathbf{F}})$, which holds here although $\mathbf{H}_s$ is singular. Each score $\mathbf{J}_t^\top(\mathbf{e}_{a_t} - \boldsymbol{\pi}_t)$ lies in $\mathrm{Range}(\mathbf{J}_t^\top \mathbf{H}_{s_t}) = \mathrm{Range}(\mathbf{J}_t^\top \mathbf{H}_{s_t} \mathbf{J}_t)$, because $\mathbf{e}_{a_t} - \boldsymbol{\pi}_t \perp \mathbf{1}$ lies in $\mathrm{Range}(\mathbf{H}_{s_t})$. A sum of PSD matrices has the sum of their ranges as range, so $\hat{\mathbf{g}} \in \mathrm{Range}(\hat{\mathbf{F}})$. So for $r \geq \operatorname{rank} \hat{\mathbf{F}}$ and $\gamma = 0$, the component of $\boldsymbol{\Theta}\tilde{\mathbf{F}}^{+}\mathbf{C}$ in $\mathrm{Range}(\hat{\mathbf{F}})$ is the full natural gradient $\hat{\mathbf{F}}^{+}\hat{\mathbf{g}}$ on the rollout. The rank, however, grows with the horizon. By Lemma 51 with $\operatorname{rank} \mathbf{H}_s = |A| - 1$,

$$\operatorname{rank} \hat{\mathbf{F}} \;\leq\; \min\big(P,\; T(|A| - 1)\big), \tag{32}$$

summed over trajectories for a batch. A sketch with $r \geq |A| - 1$ captures the Fisher of a single state exactly, not the Fisher of a rollout. For long rollouts $r < \operatorname{rank} \hat{\mathbf{F}}$, and the step is a random partial natural gradient governed by $\rho < 1$ (Proposition 8).

COMPARISON. How the methods approximate $\mathbf{F}$:

Method	Curvature	Cost and limitation
Adam, RMSProp	diagonal second moment of sampled gradients (§16.6)	no off-diagonal curvature; adapts to gradient noise rather than to $\mathbf{F}$
K-FAC, ACKTR	Kronecker-factored blocks of $\mathbf{F}$, one per layer (§20)	drops curvature between layers; needs reverse mode
TRPO	exact $\mathbf{F}$ on the Krylov subspace of m CG iterations, which contains $\mathbf{g}$ and adapts to the spectrum	about $m + 1$ Fisher-vector products, each a forward and a reverse pass, in sequence; an $O(ST)$ tape per product for a recurrent policy
SOFO-NPG	exact $\hat{\mathbf{F}}$ on a random r-dimensional subspace, $\boldsymbol{\Theta}^\top \hat{\mathbf{F}} \boldsymbol{\Theta}$ from r JVPs	r forward tangents in one parallel sweep, memory independent of T; the subspace does not contain $\mathbf{g}$; exact on $\mathrm{Range}(\hat{\mathbf{F}})$ only for $r \geq \operatorname{rank} \hat{\mathbf{F}}$

Per update, TRPO's work is about $m + 1$ forward-and-reverse passes, with $m = 10$ found sufficient and each product computed on a subsample of the batch [10, App. C], and SOFO-NPG's is about r forward passes over the batch, with r in the hundreds. SOFO-NPG therefore does more arithmetic per update; what it removes is the tape and the sequential dependence between products. Both solve the trust-region subproblem exactly within a subspace, Krylov for TRPO and random for SOFO-NPG, and a Krylov subspace of a given dimension is the better one, since it is built from $\mathbf{g}$ and $\hat{\mathbf{F}}$ (§19.1 of the optimisation notes).

5.5 SOFO-TRPO

The relative damping of (31) bounds the step only implicitly. For an explicit KL constraint, restrict TRPO's subproblem to $\mathrm{Range}(\boldsymbol{\Theta})$:

$$\max_{\mathbf{h} \in \mathbb{R}^r} \mathbf{C}^\top \mathbf{h} \quad \text{s.t.} \quad \tfrac{1}{2}\, \mathbf{h}^\top \tilde{\mathbf{F}} \mathbf{h} \leq \delta.$$

Since $\mathbf{h}^\top \tilde{\mathbf{F}} \mathbf{h} = (\boldsymbol{\Theta}\mathbf{h})^\top \hat{\mathbf{F}} (\boldsymbol{\Theta}\mathbf{h})$ exactly, the constraint is TRPO's quadratic KL model evaluated at the lifted step $\boldsymbol{\Theta}\mathbf{h}$: this is TRPO's subproblem restricted to the sketch subspace. $\tilde{\mathbf{F}}$ is singular when $r > \operatorname{rank} \hat{\mathbf{F}}$, but the gradient sketch still lies in its range:

$$\begin{aligned} &\hat{\mathbf{g}} = \hat{\mathbf{F}}^{1/2} \mathbf{w} \;\text{ for some } \mathbf{w} && (\hat{\mathbf{g}} \in \mathrm{Range}(\hat{\mathbf{F}}) = \mathrm{Range}(\hat{\mathbf{F}}^{1/2}), \text{ previous subsection}) \\ &\mathbf{C} = \boldsymbol{\Theta}^\top \hat{\mathbf{F}}^{1/2} \mathbf{w} \in \mathrm{Range}(\boldsymbol{\Theta}^\top \hat{\mathbf{F}}^{1/2}) = \mathrm{Range}(\tilde{\mathbf{F}}) && (\mathrm{Range}(\mathbf{B}\mathbf{B}^\top) = \mathrm{Range}(\mathbf{B}),\ \mathbf{B} = \boldsymbol{\Theta}^\top \hat{\mathbf{F}}^{1/2}) \end{aligned}$$

So the KL-ball derivation of §16.5 goes through with the pseudoinverse. Directions in $\mathrm{Null}(\tilde{\mathbf{F}})$ are orthogonal to $\mathbf{C}$ and do not change the objective, and the minimum-norm solution is

$$\mathbf{h}^* \;=\; \sqrt{\frac{2\delta}{\mathbf{C}^\top \tilde{\mathbf{F}}^{+} \mathbf{C}}}\; \tilde{\mathbf{F}}^{+} \mathbf{C}, \qquad \boldsymbol{\theta}_{k+1} \;=\; \boldsymbol{\theta}_k + \beta\, \boldsymbol{\Theta} \mathbf{h}^*, \tag{33}$$

with $\tilde{\mathbf{F}}^{+}\mathbf{C} = \mathbf{U}\mathbf{S}^{+}\mathbf{U}^\top \mathbf{C}$ from the eigendecomposition already computed; a damped variant uses $(\tilde{\mathbf{F}} + \gamma \mathbf{I})^{-1}$ in place of $\tilde{\mathbf{F}}^{+}$. As in TRPO, $\beta \in (0, 1]$ is found by backtracking until the sampled average KL is below δ and the surrogate improves. For a recurrent policy each trial is one forward re-run of the policy over the stored rollout, so no conjugate gradient and no backward pass are needed. TRPO's monotonic-improvement bound is for a penalty on the maximum KL over states; with the average-KL constraint used here and in TRPO, the line search is a heuristic [10, §4].

5.6 Cross-Entropy Surrogate

SOFO-NPG takes its gradient from the policy-gradient surrogate and its curvature from the Fisher, so it is not SOFO applied to a single loss. The cross-entropy (CE) surrogate is such a loss: its gradient at $\boldsymbol{\theta}_{\text{old}}$ is the policy gradient and its GGN is the Fisher.

CROSS-ENTROPY AGAINST A FIXED TARGET. For a target distribution $\mathbf{q} \in \Delta^{|A|-1}$,

$$\ell_{\text{CE}}(\boldsymbol{\phi}; \mathbf{q}) \;=\; -\sum_a q_a \log \pi_a(\boldsymbol{\phi}) \;=\; -\mathbf{q}^\top \boldsymbol{\phi} + \log \sum_a e^{\phi_a}, \qquad \nabla_{\boldsymbol{\phi}} \ell_{\text{CE}} = \boldsymbol{\pi} - \mathbf{q}, \qquad \nabla^2_{\boldsymbol{\phi}} \ell_{\text{CE}} = \mathbf{H}_s, \tag{34}$$

using $\sum_a q_a = 1$. The target enters only the linear term, so the Hessian is $\mathbf{H}_s \succeq 0$ for every $\mathbf{q}$.

ADVANTAGE-NUDGED TARGETS. From a rollout under π_{old}, with $\boldsymbol{\pi}_{\text{old},t} \coloneqq \pi_{\text{old}}(\cdot \mid s_t)$ and a scale $\alpha > 0$, define the targets and the surrogate

$$\mathbf{q}_t \;\coloneqq\; \boldsymbol{\pi}_{\text{old},t} + \alpha \hat{A}_t (\mathbf{e}_{a_t} - \boldsymbol{\pi}_{\text{old},t}), \qquad f_{\text{CE}}(\boldsymbol{\theta}) \;\coloneqq\; \frac{1}{T} \sum_{t=1}^{T} \ell_{\text{CE}}(\boldsymbol{\phi}_t(\boldsymbol{\theta}); \mathbf{q}_t), \tag{35}$$

with the $\mathbf{q}_t$ held fixed. The target moves probability towards a_t when $\hat{A}_t > 0$ and away from it when $\hat{A}_t < 0$. Then

$$\begin{aligned}
\nabla_{\boldsymbol{\phi}} \ell_{\text{CE}}\big|_{\boldsymbol{\theta}_{\text{old}}} &= \boldsymbol{\pi}_{\text{old},t} - \mathbf{q}_t = -\alpha \hat{A}_t (\mathbf{e}_{a_t} - \boldsymbol{\pi}_{\text{old},t}) && ((34), \text{ then } (35)) \\
\nabla_{\boldsymbol{\theta}} f_{\text{CE}}\big|_{\boldsymbol{\theta}_{\text{old}}} &= \frac{1}{T} \sum_t \mathbf{J}_t^\top \left(-\alpha \hat{A}_t (\mathbf{e}_{a_t} - \boldsymbol{\pi}_{\text{old},t}) \right) = -\alpha\, \hat{\mathbf{g}} && (\text{chain rule, } (24), (22)) \\
\mathbf{G}_{\text{CE}}(\boldsymbol{\theta}) &= \frac{1}{T} \sum_t \mathbf{J}_t^\top \mathbf{H}_{s_t} \mathbf{J}_t = \hat{\mathbf{F}}(\boldsymbol{\theta}) && (\text{GGN with } (34), \text{ at every } \boldsymbol{\theta})
\end{aligned}$$

At $\boldsymbol{\theta}_{\text{old}}$, SOFO on f_{CE} therefore sees $\tilde{\mathbf{G}}_{\text{CE}} = \tilde{\mathbf{F}}$ and $\mathbf{C}_{\text{CE}} = -\alpha \mathbf{C}$, with the same damping $\gamma = \lambda s_{\max}(\tilde{\mathbf{F}})$, and its step is the SOFO-NPG step (31) with learning rate $\eta\alpha$:

$$-\eta\, \boldsymbol{\Theta} (\tilde{\mathbf{F}} + \gamma \mathbf{I})^{-1} \mathbf{C}_{\text{CE}} \;=\; \eta\alpha\, \boldsymbol{\Theta} (\tilde{\mathbf{F}} + \gamma \mathbf{I})^{-1} \mathbf{C}. \tag{36}$$

VALID TARGETS. $\mathbf{q}_t$ sums to one for every α; nonnegativity constrains α. Its entries are $q_t(a) = \pi_{\text{old}}(a \mid s_t)(1 - \alpha \hat{A}_t)$ for $a \neq a_t$ and $q_t(a_t) = \pi_{\text{old}}(a_t \mid s_t) + \alpha \hat{A}_t (1 - \pi_{\text{old}}(a_t \mid s_t))$, so

$$\hat{A}_t \geq 0: \;\; \alpha \hat{A}_t \leq 1, \qquad\qquad \hat{A}_t < 0: \;\; \alpha |\hat{A}_t| \leq \frac{\pi_{\text{old}}(a_t \mid s_t)}{1 - \pi_{\text{old}}(a_t \mid s_t)}.$$

The second bound is tight for rarely chosen actions, so in practice α is small or the negative advantages are clipped.

KL READING. Since the entropy $H(\mathbf{q}_t)$ does not depend on $\boldsymbol{\theta}$,

$$f_{\text{CE}}(\boldsymbol{\theta}) \;=\; \frac{1}{T} \sum_t D_{\text{KL}}(\mathbf{q}_t \,\|\, \pi_{\boldsymbol{\theta}}(\cdot \mid s_t)) + \frac{1}{T} \sum_t H(\mathbf{q}_t)\,: \tag{37}$$

minimising f_{CE} distils the policy into the advantage-nudged targets.

WHAT THE CE SURROGATE ADDS. At $\boldsymbol{\theta}_{\text{old}}$ the two methods take the same step; they differ after it.

1. *A loss.* f_{CE} is defined at every $\boldsymbol{\theta}$ and convex in the outputs, so its GGN is PSD without supplying $\mathbf{H}_s$ by hand, and it can be monitored and line-searched. SOFO-NPG's gradient and curvature come from different objects.
2. *Several steps per rollout.* With the $\mathbf{q}_t$ fixed, K SOFO iterations on f_{CE} can follow, each with a fresh sketch and with gradient $\boldsymbol{\pi}_{\boldsymbol{\theta}_k}(\cdot \mid s_t) - \mathbf{q}_t$ and curvature $\mathbf{H}_{s_t}(\boldsymbol{\theta}_k)$ recomputed at the current iterate. The gradient shrinks as the policy approaches the targets, which bounds how far the policy moves. SOFO-NPG's gradient sketch is valid only at $\boldsymbol{\theta}_{\text{old}}$. Decreasing f_{CE} does not by itself guarantee that J increases.

With a critic for all actions, the target can instead be the Boltzmann policy $q_t(a) \propto \pi_{\text{old}}(a \mid s_t) \exp(\alpha A(s_t, a))$ of MPO [11], which uses every action's advantage rather than the sampled one's.

	SOFO-NPG	**SOFO on f_{CE}**
Loss being minimised	none	f_{CE}
Curvature	$\mathbf{H}_s$ supplied explicitly	GGN of f_{CE}, equal to $\hat{\mathbf{F}}$
Step at $\boldsymbol{\theta}_{\text{old}}$	$\boldsymbol{\Theta}(\tilde{\mathbf{F}} + \gamma \mathbf{I})^{-1} \mathbf{C}$	same, up to the factor α
Several steps per rollout	no	yes
Targets from a critic for all actions	no	yes
Constraint on α	none	$\mathbf{q}_t$ a distribution

5.7 Relation to RTRL

Real-time recurrent learning [8] propagates the sensitivity matrix $\mathbf{S}_t := \partial \mathbf{h}_t / \partial \boldsymbol{\theta}^\top \in \mathbb{R}^{S \times P}$ forward in time:

$$\mathbf{S}_t \;=\; \mathbf{D}_t \Big(\mathbf{W}_{hh}\, \mathbf{S}_{t-1} + \frac{\partial \mathbf{a}_t}{\partial \boldsymbol{\theta}^\top}\Big|_{\mathbf{h}_{t-1}\text{ fixed}} \Big), \qquad \mathbf{S}_0 = \mathbf{0}. \tag{38}$$

Multiplying on the right by $\boldsymbol{\Theta}$ gives SOFO's tangent batch,

$$\dot{\mathbf{H}}_t \;=\; \mathbf{S}_t\, \boldsymbol{\Theta}, \tag{39}$$

which obeys the tangent recursion (6). With $r = P$ and $\boldsymbol{\Theta} = \mathbf{I}_P$, SOFO's forward sweep is RTRL. RTRL gives the exact gradient online and without a tape, at $O(S^2 P)$ time and $O(SP)$ memory per step for a dense RNN. SOFO-NPG keeps r random columns and gets a sketched gradient and a sketched Fisher from them. Lam et al. prove that online RTRL with stochastic gradient steps drives the expected gradient norm to zero for a single-layer sigmoid RNN, under a contraction condition on the joint data, state and sensitivity dynamics, without a rate [12].

Element-wise recurrences make exact RTRL affordable. The recurrent Jacobian of the eLSTM is diagonal [13], and that of the recurrent trace unit (RTU) is block diagonal with 2×2 rotation-scaling blocks [14]. Each recurrent parameter then affects only its own unit, and the sensitivities cost $O(P)$ for that layer.

Method	Propagates	Gradient	Curvature	Time per step
RTRL, dense RNN	$\mathbf{S}_t \in \mathbb{R}^{S \times P}$	exact	none	$O(S^2 P)$
RTRL, eLSTM or RTU	$\mathbf{S}_t$ with diagonal or 2×2 block structure	exact for that layer	none	$O(P)$
R2AC [13]	eLSTM sensitivities in IMPALA, RMSProp	exact for the recurrent layer	none	$O(P)$
RTU with PPO [14]	RTU sensitivities, Adam, reused across PPO epochs	exact at collection time	none	$O(P)$
SOFO-NPG, dense RNN	$\mathbf{S}_t \boldsymbol{\Theta} \in \mathbb{R}^{S \times r}$	sketch $\mathbf{C}$	sketch $\tilde{\mathbf{F}}$	$O(S^2 r)$
SOFO-NPG, RTU	r tangents of the RTU	sketch $\mathbf{C}$	sketch $\tilde{\mathbf{F}}$	$O(rP)$

For a dense RNN the sketch saves a factor P/r over RTRL. For element-wise recurrences exact RTRL already costs $O(P)$ per step and r tangents cost $O(rP)$, so there the case for SOFO-NPG is the curvature, not the cost. Elelimy et al. find that RTRL traces computed during the rollout and reused across PPO epochs work better than recomputed ones, and hypothesise that staleness helps PPO stay in its trust region [14]; SOFO-TRPO enforces the KL constraint explicitly.

RTU TANGENTS. An RTU has n complex units, stored as pairs $(h^{c_1}_{t,k}, h^{c_2}_{t,k})$, with linear recurrence [14]

$$\begin{aligned} \mathbf{h}^{c_1}_t &= \mathbf{g} \odot \mathbf{h}^{c_1}_{t-1} - \boldsymbol{\varsigma} \odot \mathbf{h}^{c_2}_{t-1} + \mathbf{c} \odot \mathbf{W}^{c_1} \mathbf{x}_t, \\ \mathbf{h}^{c_2}_t &= \mathbf{g} \odot \mathbf{h}^{c_2}_{t-1} + \boldsymbol{\varsigma} \odot \mathbf{h}^{c_1}_{t-1} + \mathbf{c} \odot \mathbf{W}^{c_2} \mathbf{x}_t, \end{aligned}$$

where $g_k = r_k \cos \vartheta_k$, $\varsigma_k = r_k \sin \vartheta_k$ and $c_k = \sqrt{1 - r_k^2}$ are learned per unit, with $r_k = \exp(-\exp(\nu_k)) \in (0, 1]$ and $\vartheta_k = \exp(\vartheta^{\log}_k)$; they are constants, not gates. Differentiating along one seed, with dots for tangents,

$$\dot{\mathbf{h}}^{c_1}_t = \underbrace{\mathbf{g} \odot \dot{\mathbf{h}}^{c_1}_{t-1} - \boldsymbol{\varsigma} \odot \dot{\mathbf{h}}^{c_2}_{t-1}}_{\text{recurrence, } O(n)} + \underbrace{\dot{\mathbf{g}} \odot \mathbf{h}^{c_1}_{t-1} - \dot{\boldsymbol{\varsigma}} \odot \mathbf{h}^{c_2}_{t-1} + \dot{\mathbf{c}} \odot \mathbf{W}^{c_1} \mathbf{x}_t}_{\text{from } \dot{\nu},\, \dot{\boldsymbol{\vartheta}}^{\log}} + \underbrace{\mathbf{c} \odot \dot{\mathbf{W}}^{c_1} \mathbf{x}_t}_{O(nd)},$$

and symmetrically for c_2; a nonlinear RTU multiplies by the activation derivative. The recurrent part is element-wise, but the input term costs $O(nd)$ per seed, so r tangents cost $O(ndr) = O(rP)$ per step, as in the table.

5.8 Limitations and Open Questions

- *First order in the return.* The Fisher is a metric on policies, not the Hessian of J; that Hessian depends on the values and is indefinite away from an optimum [9, §4].
- *Advantage noise.* $\mathbf{C}$ inherits the variance of $\hat{A}_t$, so baselines and advantage estimation (§4.4 of the RL notes) are still needed.
- *Sketch size against horizon.* $\operatorname{rank} \hat{\mathbf{F}}$ grows like $T(|A| - 1)$ (32), so on long rollouts the step is a partial natural gradient with $\rho < 1$.

- *Shared sketch.* The preconditioner and the gradient sketch use the same $\boldsymbol{\Theta}$, so the stochastic-gradient theory of the optimisation notes does not apply directly (§4.5).
- *Trust region.* The average-KL line search of SOFO-TRPO, like TRPO's, has no monotonic-improvement guarantee.
- *Stacked recurrent layers.* Two stacked element-wise layers bring back cubic or quartic RTRL cost [13]; prior work cuts the gradient between layers, dropping temporal credit assignment across them [14]. SOFO's tangents pass through any depth at r times the forward cost.
- *Untested advantage over TRPO.* Each candidate advantage (memory, parallelism, an exact subspace solve, online learning) has a matching cost: more forward work per update and a random rather than a Krylov subspace (§5.4). Which wins is an empirical question.
- *No experiments.* SOFO-NPG is derived, not yet tested; candidate benchmarks are memory tasks such as DMLab, POMDP versions of MuJoCo and POPGym.

REFERENCES

[1] Youjing Yu, Rui Xia, Qingxi Ma, Máté Lengyel, and Guillaume Hennequin. Second-order forward-mode optimization of recurrent neural networks for neuroscience. In *Advances in Neural Information Processing Systems*, 2024.

[2] Aston Zhang, Zachary C Lipton, Mu Li, and Alexander J Smola. *Dive into Deep Learning*. Cambridge University Press, 2023.

[3] Nicolas Zucchet and Antonio Orvieto. Recurrent neural networks: vanishing and exploding gradients are not the end of the story, 2024.

[4] Wayne W.M. Soo, Vishwa Goudar, and Xiao-Jing Wang. Training biologically plausible recurrent neural networks on cognitive tasks with long-term dependencies. In *Advances in Neural Information Processing Systems*, volume 36, pages 32061–32074, 2023.

[5] Jonas M. Mikhaeil, Zahra Monfared, and Daniel Durstewitz. On the difficulty of learning chaotic dynamics with RNNs. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[6] David Kozak, Stephen Becker, Alireza Doostan, and Luis Tenorio. A stochastic subspace approach to gradient-free optimization in high dimensions. *Computational Optimization and Applications*, 79(2):339–368, 2021.

[7] Robert M. Gower, Dmitry Kovalev, Felix Lieder, and Peter Richtárik. RSN: Randomized subspace newton. In *Advances in Neural Information Processing Systems*, volume 32, pages 614–623, 2019.

[8] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.

[9] Sham M Kakade. A natural policy gradient. In *Advances in Neural Information Processing Systems*, volume 14, pages 1531–1538. MIT Press, 2001.

[10] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization, 2017.

[11] Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation, 2018.

[12] Samuel Chun-Hei Lam, Justin Sirignano, and Konstantinos Spiliopoulos. Convergence analysis of real-time recurrent learning (rtrl) for a class of recurrent neural networks, 2025.

[13] Kazuki Irie, Anand Gopalakrishnan, and Jürgen Schmidhuber. Exploring the promise and limits of real-time recurrent learning, 2024.

[14] Esraa Elelimy, Adam White, Michael Bowling, and Martha White. Real-time recurrent learning using trace units in reinforcement learning, 2024.